

# Star formation and streaming cosmic rays: change log

Base commit 1dd8091f8    Date 10 October 2026    Prepared for Vinod V. Pisharody

Every edit to existing PIERNIK source files made while setting up the five-variant star-forming tall box (HD, MHD, diffusive CR, two-moment CR diffusion, streaming CR), including a second round of stability safeguards for the streaming-CR halo. Each record gives the code before and after and the reason. Line numbers refer to the current files. The new problem directory `problems/tallbox_sf/` is summarised at the end.

|                                                                                         |                                                               |                                                                                    |                                                                                                          |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <div>39</div> <div>records: 11 existing source files, 1 new module, 1 new problem</div> | <div>6</div> <div>bugs fixed that broke runs or physics</div> | <div>2×</div> <div>faster streaming-CR step (4.33 → 2.20 s, case D, 20.8 pc)</div> | <div>0</div> <div>negative-<math>E_c</math> events with the new limiter (500 before, 0.5 Myr test)</div> |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|

BUG FIX

 wrong results, crash or hang

PERFORMANCE

 same results, faster

FEATURE

new behaviour

PLUMBING

 imports, namelists, declarations

### Contents

|       |                                |       |                            |
|-------|--------------------------------|-------|----------------------------|
| 1–8   | star_formation.F90             | 9–11  | timestep.F90               |
| 12–14 | initstreamingcr.F90            | 15–17 | streamingcr_source.F90     |
| 18–19 | streamingcr_transport.F90      | 20    | streamingcr_violations.F90 |
| 21–23 | timestepscr.F90                | 24–27 | interpolations.F90         |
| 28    | data_hdf5.F90                  | 29–32 | New problem: tallbox_sf    |
| 33–39 | Stability safeguards (round 2) |       | Validation                 |

## 1 New public names

PLUMBING

line 43

### BEFORE

*No such line.*

### AFTER

```
public :: field_SN, mass_SN, n_SN
```

**WHY** The `tallbox_sf` problem calls the new `field_SN` routine (record 8) and needs `mass_SN` and `n_SN` to turn the Kennicutt–Schmidt rate into a number of SNe.

## 2 Default of `max_part_mass`

BUG FIX

lines 86 and 116

### BEFORE

```
max_part_mass = mass_SN * n_SN ! Maximum mass of sink particles
```

### AFTER

```
max_part_mass = -1.0          ! sentinel <=0 means mass_SN * n_SN
...
! after compare_namelist:
if (max_part_mass <= 0.0) max_part_mass = mass_SN * n_SN
```

**WHY** The old default was computed from the default `mass_SN` and `n_SN` before the namelist was read. Setting either in `problem.par` left `max_part_mass` at the old value, and that mismatch left `mfdv` undefined (record 4).

### 3 SN energy split in SF

BUG FIX

lines 231-240

#### BEFORE

```
#ifndef COSM_RAYS
    en_SN01 = cr_eff*cr_active*en_SN
    en_SN09 = (1-cr_eff*cr_active)*en_SN
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif
#ifndef STREAM_CR
    en_SN01 = scr_eff*en_SN
    en_SN09 = (1-scr_eff)*en_SN
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif
```

#### AFTER

```
#if defined(COSM_RAYS)
    en_SN01 = cr_eff*cr_active*en_SN
    en_SN09 = (1-cr_eff*cr_active)*en_SN
#elif defined(STREAM_CR)
    en_SN01 = scr_eff*en_SN
    en_SN09 = (1-scr_eff)*en_SN
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif
```

**WHY** In a COSM\_RAYS-only build the `#else` branch of the STREAM\_CR block reset the CR share to zero, so SNe from the instantaneous feedback path injected no cosmic rays at all.

### 4 mfdv in delayed feedback

BUG FIX

lines 393-395

#### BEFORE

```
if (mass_SN_tot .eq. max_part_mass) then
    if (aint(pset%pdata%mass / mass_SN_tot) .gt. 1) &
        print *, 'WARNING: ...'
    mfdv = 1 / cg%dvol
endif
```

#### AFTER

```
if ((mass_SN_tot .eq. max_part_mass) .and. &
    (aint(pset%pdata%mass / mass_SN_tot) .gt. 1)) &
    & print *, 'WARNING: ...'
mfdv = 1 / cg%dvol
```

**WHY** `mfdv` was set only inside the `if`. Whenever `mass_SN_tot` differed from `max_part_mass` it was undefined and the injected SN energy was garbage. This is a likely cause of the negative `sqrt` argument guarded against in commit 1dd8091f8. The warning is kept.

## 5 Converging-flow criterion under STREAM\_CR

BUG FIX

lines 635–637

### BEFORE

```
#endif /* COSM_RAYS */
#ifdef STREAM_CR
    if ((divv_crit)) return    ! convergent flow.
                                ! Need to also account for divv.
#endif /* STREAM_CR */
```

### AFTER

```
#else /* !COSM_RAYS */
    if ((divv_crit) .and. &
        (local_divv(cg, pfl, i, j, k) .ge. 0)) return
#endif /* !COSM_RAYS */
```

**WHY** With STREAM\_CR and the default `divv_crit = .true.`, `SF_crit` rejected every cell, so no stars ever formed. Builds without COSM\_RAYS (HD, MHD and STREAM\_CR) have no `divvel` field, so the divergence is now computed locally (record 6).

## 6 New function `local_divv`

FEATURE

lines 657–677

### BEFORE

*Not present.*

### AFTER

```
real function local_divv(cg, pfl, i, j, k) result(divv)
    divv = 0.0
    if (dom%has_dir(xdim)) divv = divv + half*cg%idx * &
        (u(imx,i+1)/u(idn,i+1) - u(imx,i-1)/u(idn,i-1))
    ! same for y and z (shortened here)
end function local_divv
```

**WHY** Centred-difference  $\nabla \cdot \mathbf{v}$  of the given fluid, using only active directions. Needed by record 5.

## 7

Energy split and CR injection in **TIGRESS\_injection**

BUG FIX

lines 1106-1116

## BEFORE

```

#ifdef COSM_RAYS
    en_SN01 = ... ; en_SN09 = ...
    if (cr_active > 0) call sf_inject(..., 0.0, &
        mfdv*en_SN01*frac, ...)
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif
#ifdef STREAM_CR
    en_SN01 = ... ; en_SN09 = ...
    call sf_inject(..., 0.0, mfdv*en_SN01*frac, ...)
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif

```

## AFTER

```

#if defined(COSM_RAYS)
    en_SN01 = ... ; en_SN09 = ...
#elif defined(STREAM_CR)
    en_SN01 = ... ; en_SN09 = ...
#else
    en_SN01 = 0.0 ; en_SN09 = en_SN
#endif
if (en_SN01 > 0.0) call sf_inject(..., 0.0, &
    mfdv*en_SN01*frac, ...)

```

**WHY** Same overwrite as record 3. With COSM\_RAYS only, the CRs were injected before the reset and the thermal or kinetic part then used the full  $E_{\text{SN}}$ , so each SN delivered  $(1 + \text{cr\_eff}) \cdot E_{\text{SN}}$ . After the fix each SN gives  $0.1 \times 10^{51}$  erg to CRs: measured  $5.23 \times 10^6$  (C1) and  $5.14 \times 10^6$  (D) code units against  $5.26 \times 10^6$  expected.

## 8 New subroutine `field_SN(pos)`

FEATURE

lines 1176–1292

### BEFORE

*Not present.*

### AFTER

```
subroutine field_SN(pos)    ! collective, same pos on all ranks
  ! snap pos to a cell centre
  ! pass 1: cells within 3 dx (periodic minimum image)
  !           -> ambient density, cell count (1 Allreduce)
  ! pass 2: call TIGRESS_injection(...) per cell
  !           update SNe_n and sne_cnt
end subroutine field_SN
```

**WHY** Provides SILCC-style random SN driving. Reusing `TIGRESS_injection` keeps the CR split and the Kim & Ostriker 2015 regimes identical to star-particle SNe. No particle-to-rank MPI exchange is needed because every rank knows the position.

## timestep.F90 `src/base/timestep.F90`

## 9 Imports

PLUMBING

lines 145–146

### BEFORE

```
use timestepscr,      only: timestep_scr
use initstreamingcr, only: dt_scr, nsub, nsub_scr, &
                        scr_negative
```

### AFTER

```
use timestepscr,      only: timestep_scr, update_cred
use initstreamingcr, only: dt_scr, nsub, nsub_scr, &
                        scr_negative, scr_redo_on_violation
```

**WHY** Needed for records 10 and 11.

## 10 Reduced speed of light set before the CR timestep

FEATURE

line 246

### BEFORE

*Not present.*

### AFTER

```
if (.not. scr_redo_on_violation) call update_cred
```

**WHY** In the new scheme  $c_{\text{red}}$  follows the signal speed when  $dt$  is chosen (record 22) instead of being raised by redoing steps.

## 11 Adaptive sub-cycling ( `nsub = 0` )

BUG FIX + PERFORMANCE

lines 254–257

### BEFORE

```
if (dt < dt_scr) then
  scr_negative = .true.
  dt_scr = dt
  nsub_scr = I_TWO
else
  nsub_scr = max(I_ONE, ceiling(dt/dt_scr))
  if (mod(nsub_scr, 2) /= 0) nsub_scr = nsub_scr + I_ONE
  dt_scr = dt/nsub_scr
endif
```

### AFTER

```
nsub_scr = max(I_ONE, ceiling(dt/dt_scr))
dt_scr = dt/nsub_scr
```

**WHY** Three problems. (a) When the MHD step was already shorter than the CR step it flagged a violation, and every redo shrank  $dt$  further, so the run looped until `max_redostep_attempts` aborted it. (b) That branch advanced the CRs by  $2 \cdot dt$  per half-step instead of  $dt$ . (c) Rounding up to an even count wasted up to about 40% of the CR work ( $2.4 \rightarrow 4$  sub-cycles at 20.8 pc). Fixed-`nsub` runs are unchanged and stay bit-identical; adaptive runs differ by about  $10^{-4}$  with no loss of accuracy (see Validation).

## 12 New namelist switches

FEATURE

lines 70-71

### BEFORE

*Not present.*

### AFTER

```
logical :: scr_redo_on_violation ! .true.: legacy redo
                                ! .false.: clip locally
logical :: scr_fallback_1st      ! first-order faces where
                                ! reconstruction is unphysical
```

**WHY** Turn the new causality handling and the first-order fallback on or off, and keep the original redo behaviour selectable.

## 13 Safeguard counters

FEATURE

lines 90-92

### BEFORE

*Not present.*

### AFTER

```
integer :: scr_nclip_src = 0
integer :: scr_nclip_face = 0
integer :: scr_nfallback = 0
```

**WHY** Show how often the local safeguards act. The tallbox\_sf problem writes them to the tsl file and resets them at each dump.

## 14 Namelist, defaults and broadcast

PLUMBING

lines 117-118, 140-141, 186-187, 191, 233-234

### BEFORE

```
... transport_scheme, cred_max
...
nl = 4
```

### AFTER

```
... transport_scheme, cred_max, &
    scr_redo_on_violation, scr_fallback_1st
scr_redo_on_violation = .false.
scr_fallback_1st      = .true.
lbuff(5) = scr_redo_on_violation
lbuff(6) = scr_fallback_1st
nl = 6
! slaves unpack lbuff(5:6)
```

**WHY** Standard PIERNIK namelist plumbing for record 12.

**streamingcr\_source.F90** `src/fluids/cosmicrays_streaming/streamingcr_source.F90`

## 15 Imports and locals

PLUMBING

lines 50, 68

### BEFORE

```
& scr_causality_limit
real :: gpcx, gpcy, gpcz, ec_source_, e_feed
```

### AFTER

```
& scr_causality_limit, scr_redo_on_violation, scr_nclip_src
real :: gpcx, gpcy, gpcz, ec_source_, e_feed, fmag, fscale
```

**WHY** Needed by record 16.

## 16 Causality check in the source step

BUG FIX

lines 186-200 (old line 191 removed)

### BEFORE

```
! new Ec and Fc stored first, then:
if (sqrt(newf1*newf1 + newf2*newf2 + newf3*newf3) &
    > scr_causality_limit * newec) scr_negative = .true.
```

### AFTER

```
fmag = sqrt(newf1*newf1 + newf2*newf2 + newf3*newf3)
if (fmag > scr_causality_limit * newec) then
  if (scr_redo_on_violation) then
    scr_negative = .true.          ! legacy
  else
    fscale = scr_causality_limit * newec / fmag
    newf1 = newf1*fscale
    newf2 = newf2*fscale
    newf3 = newf3*fscale
    scr_nclip_src = scr_nclip_src + 1
  endif
endif
! then Ec and Fc are stored
```

**WHY** This check caused the case D aborts. Next to a fresh SN the front saturates at  $|F_c|/(c_{\text{red}}E_c) \approx 1.03$ , and in cells where  $E_c$  went negative and was floored  $|F_c| \gg E_c$ . Neither ratio depends on  $c_{\text{red}}$ , so redoing the step with a larger  $c_{\text{red}}$  could never succeed.

## 17 Index lookup in `update_gradpc_here`

PERFORMANCE

lines 236, 258-261

### BEFORE

```
cg%w(wna%ind(gpcn))%arr = 0.0
T => cg%w(wna%ind(gpcn))%arr
```

### AFTER

```
gpci = wna%ind(gpcn)
cg%w(gpci)%arr = 0.0
T => cg%w(gpci)%arr
```

**WHY** Inside a whole-array assignment gfortran re-evaluated the `wna%ind` string search for every element: about 92 million calls in 20 steps. Results are bit-identical.

# streamingcr\_transport.F90

src/fluids/cosmicrays\_streaming/streamingcr\_transport.F90

## 18 Index lookup in `advance_scr`

PERFORMANCE

lines 71-75

### BEFORE

```
cg%w(wna%ind(uh_n))%arr(:,:,:,) = cg%u(:,:,:,)
```

### AFTER

```
uhi = wna%ind(uh_n)  
cg%w(uhi)%arr(:,:,:,) = cg%u(:,:,:,)
```

**WHY** Same per-element lookup as record 17, about 69 million calls in 20 steps. Bit-identical.

## 19 Index lookup in `update_scr_fluid`

PERFORMANCE

lines 106, 117, 138

### BEFORE

```
! inside the loop over 1D lines:  
vdiff => cg%w(wna%ind(v_dfst))%get_sweep(ddim, i1, i2)
```

### AFTER

```
vdfi = wna%ind(v_dfst)           ! once, before the loops  
...  
vdiff => cg%w(vdfi)%get_sweep(ddim, i1, i2)
```

**WHY** The lookup ran once per 1D line, about 1.1 million times in 20 steps. Bit-identical.

## streamingcr\_violations.F90

src/fluids/cosmicrays\_streaming/streamingcr\_violations.F90

### 20 Signal-speed check only in legacy mode

FEATURE

lines 70, 94–96

#### BEFORE

```
! always: compute max(|u| + c_f) and flag a violation
! when cred / max < cred_to_mhd_threshold
```

#### AFTER

```
! Without the legacy redo, cred follows the signal
! speed when dt is chosen, nothing to check here
if (.not. scr_redo_on_violation) return
```

**WHY** The same criterion is now applied before the step by setting  $c_{\text{red}}$  (record 22). Checking it again after the step would only trigger redos.

## timestepscr.F90

src/fluids/cosmicrays\_streaming/timestepscr.F90

### 21 Public list

PLUMBING

line 40

#### BEFORE

```
public :: timestep_scr, scr_on_success, scr_on_violation
```

#### AFTER

```
public :: timestep_scr, scr_on_success, scr_on_violation, &
        update_cred
```

**WHY** Called from `timestep.F90` (record 10).

## 22 New subroutine `update_cred`

FEATURE

lines 94–157

### BEFORE

*Not present.*

### AFTER

```
! over all leaf cells:
!   umax = max(|u| + c_f),   c_f^2 = c_s^2 + v_A^2 + c_CR^2
call piernik_MPI_Allreduce(umax, pMAX)
cred = min(cred_max, max(cred_min,
                        cred_to_mhd_threshold * umax,
                        cred_decay_fac * cred))
```

**WHY** The core of the  $c_{\text{red}}$  rework. It applies the existing threshold criterion when  $dt$  is chosen, so it already sees the feedback injected at the end of the previous step.  $c_{\text{red}}$  grows at once and decays by at most `cred_decay_fac` per step, and no step is redone because of it.

## 23 `scr_on_violation` and `scr_on_success`

BUG FIX

lines 163–169, 196–200

### BEFORE

```
! always modified cred:
! on violation: cred = cred * cred_growth_fac
! on success:   cred = max(cred_floor_dyn, cred_min)
```

### AFTER

```
! first line of both routines:
if (.not. scr_redo_on_violation) return
```

**WHY** Otherwise `scr_on_success` would overwrite the signal-speed  $c_{\text{red}}$  after every step, and a redo caused only by the hydro CFL would wrongly double  $c_{\text{red}}$ .

# interpolations.F90, interpol\_scr

src/scheme/Riemann\_Solver/interpolations.F90

## 24 Imports and locals

PLUMBING

lines 143, 153

### BEFORE

```
use initstreamingcr, only: cred
```

### AFTER

```
use initstreamingcr, only: cred, scr_fallback_1st, &  
    smallescr, scr_causality_limit, scr_nfallback  
integer :: i, j, b
```

**WHY** Needed by records 25 to 27.

## 25 First-order fallback at bad faces

FEATURE

lines 157-170

### BEFORE

*Reconstructed face states were always used.*

### AFTER

```
if (scr_fallback_1st) then  
  do j = 1, scrind%nscr ; b = 4*(j-1) + 1  
    do i = 1, size(ql, 1)  
      if (bad_state(ql(i, b:b+zdim)) .or. &  
          bad_state(qr(i, b:b+zdim))) then  
        ql(i, b:b+zdim) = u(i, b:b+zdim)  
        qr(i, b:b+zdim) = u(i+1, b:b+zdim)  
        scr_nfallback = scr_nfallback + 1  
      endif  
    enddo  
  enddo  
endif
```

**WHY** Where the second-order reconstruction gives  $E_c$  below the floor or a superluminal flux, typically at sharp SN fronts, that face uses first-order states, which HLLC handles robustly. Added as an extra safeguard. (Vinod's suggestion was different: a more diffusive Riemann solver at such faces. It was implemented, tested and rejected; see the Validation section. Its robust replacement is the positivity-preserving limiter, record 34.)

## 26 New helper `bad_state`

FEATURE

lines 178–186

### BEFORE

*Not present.*

### AFTER

```
bad_state = (q(1) < smallescr) .or. &
             (sum(q(2:4)**2) > (scr_causality_limit*cred*q(1))**2)
```

**WHY** The test used by record 25 for one face state ( $E_C$ ,  $F_X$ ,  $F_Y$ ,  $F_Z$ ).

## 27 Face clip in `limit_face`

BUG FIX

lines 190, 213–227

### BEFORE

```
cap = max(0.0, cred * ec)
if (fc2 > scr_causality_limit**2 * cap**2) then
  scr_negative = .true.
  scale = cap / sqrt(fc2) ! clip to 1.0*cred*Ec
  ...
  if (master) call warn('... causality violated ...')
```

### AFTER

```
cap = max(0.0, scr_causality_limit * cred * ec)
if (fc2 > cap * cap) then
  scale = cap / sqrt(fc2) ! clip to limit*cred*Ec
  ...
  scr_nclip_face = scr_nclip_face + 1
  if (scr_redo_on_violation) then
    scr_negative = .true.
    if (master) call warn(...)
  endif
```

**WHY** Face states were already clipped and then a global redo was forced anyway. Now the redo and the warning, which used to print every time, happen only in legacy mode. One small semantic change: the clip target is now  $\text{limit} \cdot c_{\text{red}} \cdot E_C$  (0.8 by default) instead of exactly  $c_{\text{red}} \cdot E_C$ , which matches the source-step clip in record 16.

## 28 Streaming-velocity output `vel_stream_x/y/z`

BUG FIX

lines 566, 572, 578

### BEFORE

```
tab(:, :, :) = cg%b(xdim, RNG) / &  
               sqrt(cg%u(fl_dni%idn, RNG)) * ...
```

### AFTER

```
tab(:, :, :) = cg%b(xdim, RNG) / &  
               sqrt(cg%u(flind%ion%idn, RNG)) * ...  
               ! Alfven speed always w.r.t. ions
```

**WHY** `fl_dni` is set only for variable names with a fluid suffix (for example `deni`). For `vel_stream_*` it was a null pointer and writing the first snapshot crashed with a segfault. The Alfvén speed always uses the ion density.

## New problem: tallbox\_sf problems/tallbox\_sf/ (new directory)

## 29 `piernik.def`

NEW FILE

problems/tallbox\_sf/piernik.def

**WHAT** Defines IONIZED, THERM, NBODY (which implies SELF\_GRAV and MULTIGRID) and RANDOMIZE. The physics variant is chosen with `./setup -d`, with the dummy define `H5PFC_EATS_1ST` first, because the local `h5pfc` wrapper drops its first argument and silently removed `-DSTREAM_CR` or `-DCOSM_RAYS`.

### 30 `initproblem.F90`

[NEW FILE](#)

`problems/tallbox_sf/initproblem.F90`

- WHAT**
- mcrwind-like initial condition: one common  $\rho(z)$  from `csim2 = cs2(T0) · (1 + alpha + beta_eff)`, horizontal B with  $P_{\text{mag}}/P_{\text{gas}} = \alpha$ , a small CR seed, Ferrière potential reused from tallbox.
  - TIGRESS-style random velocity field: Fourier modes drawn on the master and broadcast, per-mode Helmholtz projection,  $|v_k|^2 \propto k^{-4}$ , normalised to a mass-weighted  $v_{\text{rms}}$ .
  - SILCC-style field-SN driver at the Kennicutt–Schmidt rate: Poisson numbers, random and density-peak positions, tapered after `t_drive_end`, exploded through `field_SN`.
  - Extra `tsl` columns: SFR over 10 and 40 Myr, stellar mass, scale height,  $\sigma_v$ , outflow rates,  $E_{\text{CR}}$ , field-SN count, and for `STREAM_CR` the current  $c_{\text{red}}$  and the clip and fallback counters.
  - SN rate and counters saved as restart attributes.

### 31 `problem.par.{A,B,C1,C2,D}`

[NEW FILES](#)

`problems/tallbox_sf/`

- WHAT**
- One parameter file per variant at 20.8 pc on a  $1 \times 1 \times 4$  kpc box ( $48 \times 48 \times 192$ ). `solver_str = 'riemann'` because star formation only runs in the split solver; `dt_max = 0.005`; `d0 = 0.031` for  $\Sigma_{\text{gas}} \approx 10 \text{ M}_{\odot}/\text{pc}^2$ ; CR diffusion  $\kappa = 10^5 \text{ pc}^2/\text{Myr} \approx 3 \times 10^{28} \text{ cm}^2/\text{s}$  in both CR modules; `cred_min = 1e4` for C2 and D.

### 32 Supporting files

[NEW FILES](#)

`info`, `build_and_run.sh`, `analysis/compare.py`, `cool_Kobayashi.txt`

- WHAT**
- Documentation (`info`), a build-and-launch script for other machines (`build_and_run.sh` `<20|10|8> <nproc> [cases]`), comparison plots (`compare.py`: time series, vertical profiles, slices, phase diagrams) and a copy of the cooling table.

## Stability safeguards (round 2) `streaming CR transport, Alfvén-speed ceiling`

Motivated by crashes in the low-density halo: a large sound speed and a distorted field drive  $v_A$  up, gas is evacuated, cells hit the floor, and the feedback loop ends in a crash. All new switches default to off, so existing setups stay bit-identical.

### 33 CR time-step factor `cfl_scr`

FEATURE

timestepscr.F90:89; initstreamingcr.F90:73, 122, 147, 187, 236

#### BEFORE

```
dt = dt_scr      ! cfl_scr not necessary because we check
                  ! violation directly on dt_scr and cred
```

#### AFTER

```
dt = cfl_scr * dt_scr
! namelist STREAMING_CR: cfl_scr (default 1.0)
```

**WHY**  $dt_{scr} = \Delta x / c_{red}$  with waves at  $c_{red}/\sqrt{3}$  gives a summed 3D Courant number of  $\sqrt{3} \approx 1.73$  in an unsplit update, above the stable limit of 1. `cfl_scr = 1/√3` restores it; in the test it cut negative- $E_c$  events by about 40% at about 45% extra cost. The default of 1.0 keeps the original step.

### 34 Positivity-preserving limiter on the $E_c$ flux

FEATURE

streamingcr\_transport.F90:162–167, 314–355; initstreamingcr.F90:72, 146, 195, 244

#### BEFORE

*HLLC fluxes were applied unchanged; negative  $E_c$  was reset to the floor afterwards, creating CR energy.*

#### AFTER

```
! per face, E0 = step-start energy, lam = rk_coef*dt_scr/dx
fmax = max(E0_L, 0) / (2*nd*lam)
fmin = -max(E0_R, 0) / (2*nd*lam)
if (F_HLLE outside [fmin, fmax]) then
  F_lo = first-order HLL Ec flux, speeds +-cred
  theta = largest value keeping theta*F_HLLE
           + (1-theta)*F_lo inside [fmin, fmax]
  F = clamp(theta*F_HLLE + (1-theta)*F_lo, fmin, fmax)
endif
! namelist STREAMING_CR: scr_pp_limiter (default .false.)
```

**WHY** Guarantees that transport cannot drive  $E_c$  negative (after Hu, Adams & Shu 2013). Conservative, because both neighbours see the same flux. Only the  $E_c$  flux is limited, and the target is  $E \geq 0$ , not the floor: a first version that limited all four components towards the floor froze the whole halo and was unstable at `cfl_scr = 1`. In the final test it acted on about  $10^4$  faces in 0.5 Myr, removed all negative- $E_c$  events and cost no measurable time.

## 35 Diagnostics: limited faces and negative $E_c$

FEATURE

```
initstreamingcr.F90:95-96; streamingcr_source.F90:50, 127
```

### BEFORE

*Not present.*

### AFTER

```
integer :: scr_npp      = 0    ! faces changed by the limiter
integer :: scr_nfloor = 0    ! cells transport drove negative
...
if (ec < 0.0) scr_nfloor = scr_nfloor + 1
```

**WHY** Negative  $E_c$  after transport is the event behind the blow-ups, so it is counted (tsl column `nfloor`) together with the limiter activity (`npp_limited`). A first version counted cells below the floor, which is useless because the halo sits at the floor permanently.

## 36 Namelist parameter `va_max`

PLUMBING

```
global.F90:50, 82, 122, 242, 351, 409
```

### BEFORE

*Not present.*

### AFTER

```
real :: va_max    ! Alfven-speed ceiling (0: off)
! added to NUMERICAL_SETUP, default 0.0,
! broadcast as rbuff(18)
```

**WHY** Switch for the Alfvén-speed ceiling (record 37), placed beside `smallld` because it is a floor-type safeguard.

### 37 New module `alfven_limiter`

[NEW FILE](#)

`src/magnetic/alfven_limiter.F90`

#### BEFORE

*Not present.*

#### AFTER

```
rho_min = 2*emag / va_max**2      ! v_A^2 = B^2/rho
if (rho < rho_min) then
  f = rho_min / rho
  rho = rho_min
  mom = f * mom                    ! keep velocity
  ien = f * (ien - emag) + emag    ! keep temperature
  va_added_mass = va_added_mass + (rho_min - rho_old)*dvol
endif
```

**WHY** Breaks the halo runaway described above by never letting  $v_A$  exceed `va_max`. The added mass is counted so it can be monitored (tsl columns `va_added_mass`, `va_ncells`).

### 38 Ceiling applied once per step

[PLUMBING](#)

`fluidupdate.F90:70, 95`

#### BEFORE

```
end select
call ppp_main%stop(fu_label)
```

#### AFTER

```
end select
#ifdef MAGNETIC
call limit_alfven_speed ! no-op unless va_max > 0
#endif
call ppp_main%stop(fu_label)
```

**WHY** Runs after every solver type (split and unsplit) once per full step.

### 39 `tallbox_sf` settings and tsl columns

[PROBLEM FILES](#)

`problems/tallbox_sf/initproblem.F90`, `problem.par.{B,C1,C2,D}`

**WHAT** New tsl columns: `va_added_mass`, `va_ncells` (magnetic runs) and `npp_limited`, `nfloor` (streaming-CR runs). Parameter files: `va_max = 1000` pc/Myr in B, C1, C2 and D (about 70× the disk value, so it only catches a runaway); `scr_pp_limiter = .true.` and `cfl_scr = 1.0` in C2 and D.

**Validation** `scr_test` suite + `tallbox_sf` case D, 6 MPI ranks

Baseline binaries were built from the code before records 11 and 17–19, and every HDF5 dataset and tsl file was compared after each change. scr\_test 2, 5 and 6 fail on the unchanged code too (old parameter name `solver_type` ; old fluid-index API `flind%nscri` ) and were skipped.

| Test                                   | Records 17–19 (index lookups) | Record 11 (sub-cycling)                                                           |
|----------------------------------------|-------------------------------|-----------------------------------------------------------------------------------|
| scr_test 1, 3, 4, 7, 9                 | bit-identical                 | bit-identical                                                                     |
| scr_test 8 (512 <sup>2</sup> CR blast) | bit-identical                 | bit-identical                                                                     |
| scr_test 8 with <code>nscri = 0</code> | bit-identical                 | density L1 $\leq 9 \times 10^{-5}$ , $E_c$ L1 $\leq 4 \times 10^{-4}$ , no growth |
| tallbox_sf D, 0.1 Myr                  | bit-identical                 | gas L1 $10^{-8}$ – $10^{-7}$ , $E_c$ L1 $\approx 10^{-3}$                         |

Accuracy check for record 11: against a reference run that steps exactly at the CR limit ( `nscri = -1` ), the old and new adaptive runs are equally close (density L1  $9.3 \times 10^{-4}$  vs  $9.8 \times 10^{-4}$ ;  $E_c$  L1  $6.7 \times 10^{-3}$  vs  $6.8 \times 10^{-3}$ ). They differ from each other about ten times less than either differs from the reference.

| Run                                    | Before | After records 17–19 | After record 11 too |
|----------------------------------------|--------|---------------------|---------------------|
| scr_test 8, total wall time            | 1539 s | 789 s               | 788 s               |
| scr_test 8 with <code>nscri = 0</code> | 1179 s | 479 s               | 464 s               |
| tallbox_sf D, wall time per step       | 4.33 s | 2.72 s              | 2.20 s              |

Timings on an AMD Ryzen 5 4600H (6 physical cores). Running 12 ranks on its 12 hyperthreads gave no speed-up over 6.

Stability safeguards: case D, 0.5 Myr, same seeds

| Configuration                                                      | Negative $E_c$ | Flux clips (source / face) | Cells at floor | $E_{CR}$ (code) | Wall time |
|--------------------------------------------------------------------|----------------|----------------------------|----------------|-----------------|-----------|
| Before round 2<br>( <code>cfl_scr</code> 1.0)                      | 500            | 1,408 / 1,364              | 22%            | 1.9914e8        | 117 s     |
| <code>cfl_scr</code> 0.577                                         | 295            | 2,334 / 3,850              | 22%            | 1.9913e8        | 169 s     |
| HLL flux fallback at floor faces, 0.577<br>(rejected)              | 315–424        | ~2,300 / ~23,000           | 13%            | 1.9914e8        | 168 s     |
| HLL fallback at sharp jumps, 0.577<br>(rejected)                   | 14,000–16,000  | ~25,000 / ~34,000          | 22%            | 2.003–2.008e8   | 169 s     |
| <b>Positivity limiter,</b><br><code>cfl_scr</code> 1.0<br>(chosen) | <b>0</b>       | 2,265 / 3,170              | 22%            | 1.9908e8        | 119 s     |

The HLL fallback at `cfl_scr` 1.0 crashed outright (negative gas internal energy). At floor faces it acted on about  $10^9$  faces, effectively the whole halo, spreading CRs at  $c_{red}/\sqrt{3}$  regardless of  $\kappa$ . At sharp jumps, mixing it with HLL next door created oscillations and spurious CR energy. With the limiter,  $E_{CR}$  is 0.03% lower than before because the floor resets no longer add energy.

Alfvén-speed ceiling: case B, 0.5 Myr

| Setting                                                                                      | Result                                                                                                      |
|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>va_max</code> = 8 pc/Myr (below the initial $v_A \approx 11.7$ , so active everywhere) | <code>vai_max</code> held at 8.000; added mass $1.2534 \times 10^7 M_\odot$ equals the change in total mass |
| <code>va_max</code> = 300 pc/Myr (never triggered)                                           | <b>bit-identical to the run without a ceiling, tsl included</b>                                             |
| All new switches at their defaults                                                           | <b>bit-identical HDF5 output in <code>scr_test 1, 3, 4, 7, 9</code> and <code>tallbox_sf D</code></b>       |

Generated from `git diff` of the working tree against commit 1dd8091f8. Nothing in this log has been committed yet.